Design and apply context-aware access Controls based on user identity, device Posture, location and application risk.

Stage-1

Title of the project :

Design and apply context-aware access controls based on user identity.

Overview :

Context-aware access control is a security approach that allows or denies access to resources based not only on the user's identity but also on the context of the access request. The context may include factors such as the user's location, device type, time of access, network condition, and behavior. This method improves security by ensuring that users can access systems only when their identity and surrounding conditions are verified and considered safe. The design of context-aware access control begins with user identity verification. Every user must have a unique identity, which is authenticated using methods such as passwords, biometric verification, or multi-factor authentication.

After authentication, the system evaluates the context information, such as whether the user is logging in from a trusted device, the geographic location, the time of the request, and the type of network being used. Once the context is analyzed, the system applies access control policies. These policies define what resources a user can access under specific conditions. For example, an employee may be allowed to access company data only during working hours or only from an authorized device. If the context does not meet the defined policy rules, the system may restrict access, request additional authentication, or block the request entirely.

Implementing context-aware access control improves system security, flexibility, and user management. It reduces the risk of unauthorized access and helps organizations protect sensitive information while still allowing legitimate users to work efficiently. This approach is widely used in modern cloud systems, enterprise networks, and Internet of Things (IoT) environments.

List of teammates:

S.NO	NAME	COLLEGE	CONTACT
1	MEENA P	HINDUSTHAN COLLEGE OF SCIENCE AND COMMERCE, Ingur, Perundhurai 638052.	meenapalani5040@gmail.com
2	SADHANA.M	HINDUSTHAN COLLEGE OF SCIENCE AND COMMERCE, Ingur, Perundhurai 638052	sadhana4665@gmail.com
3	KUZHANDHAIMERY M	HINDUSTHAN COLLEGE OF SCIENCE AND COMMERCE, Ingur, Perundhurai 638052	kuzhandhaimerygmail.com
4	NIVI N	HINDUSTHAN COLLEGE OF SCIENCE AND COMMERCE, Ingur, Perundhurai 638052	nivisweety2005@gmail.com

List of Vulnerability Table :

S.NO	VULNERABILITY NAME	CWE/CVE – No
1.	Broken Access Control	CWE-65
2.	Security Misconfiguration	CWE-16
3	Software Supply Chain Failures	CWE-447
4	Cryptographic Failures	CWE-334
5	Injection	CWE-114
6	Insecure Design	CWE-1125
7	Authentication Failures	CWE-1393
8	Software or Data Integrity Failures	CWE-506
9	Security Logging and Alerting Failures	CWE-778
10	Mishandling of Exceptional Conditions	CWE-369

REPORT:

VULNERABILITY NAME: Windows Hard Link

CWE/CVE : CWE-65

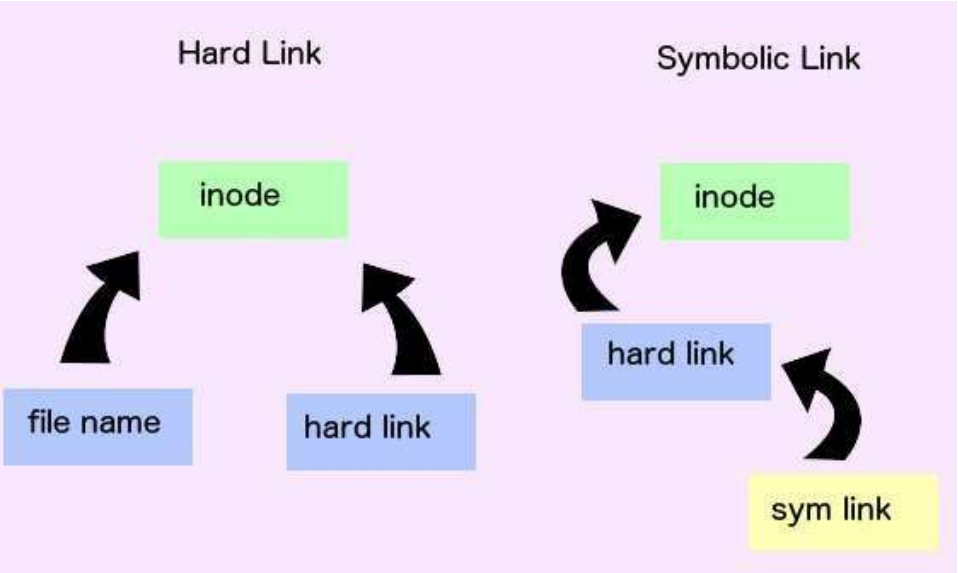
OWASP/SANS Category: A01:2025 – Broken Access Control

Description:

A Windows hard link is a feature of the NTFS file system that allows multiple file names or paths to point to the same physical file data on a disk. Instead of creating a separate copy of the file, a hard link creates an additional directory entry that references the same data. This means the file content is shared, so any changes made through one hard link are immediately reflected when accessing the file through any other link. Hard links do not use extra disk space because they reference the same file data. The file is only permanently deleted when all hard links pointing to it are removed. However, hard links can only be created for files (not folders) and must exist on the same drive or NTFS volume.

Business Impact:

The business impact of Windows hard links mainly relates to storage efficiency, system performance, and data management. Hard links allow multiple file names or paths to reference the same physical file without creating duplicate copies. This helps organizations save disk space, especially when the same files are needed in multiple locations, such as application resources, shared documents, or backup systems. Because hard links avoid redundant data storage, they can reduce storage costs and improve disk utilization in enterprise environments.Hard links can also improve system performance and efficiency by reducing the time required to copy or duplicate large files, since only a new reference is created rather than a full file copy. Additionally, they support better file organization by allowing the same data to appear in different directories without duplication. However, there are potential risks if administrators are not aware of hard links, because deleting a file in one location may not actually remove the data if other hard links still exist, which can lead to data management or security concerns. Proper monitoring and policies are therefore important to ensure hard links are used safely in business systems.



Recommendations:

1. Use Hard Links to Save Disk Space:

Hard links are recommended when you want multiple filenames for the same file without creating duplicate copies. Since all links reference the same data, it helps reduce storage usage.

2. Use Within the Same NTFS Volume:

Hard links work only within the same NTFS file system. It is recommended to create them inside the same drive or partition, because they cannot link files across different drives.

3. Use for File Organization:

Hard links are useful when the same file needs to appear in multiple directories. Instead of copying the file, a hard link can be used to access the same data from different locations.

4. Avoid Using for Directories:

In Windows, it is recommended not to create hard links for directories because it can cause file system complexity and potential loops.

VULNERABILITY NAME: Configuration

CWE/CVE : CWE-16

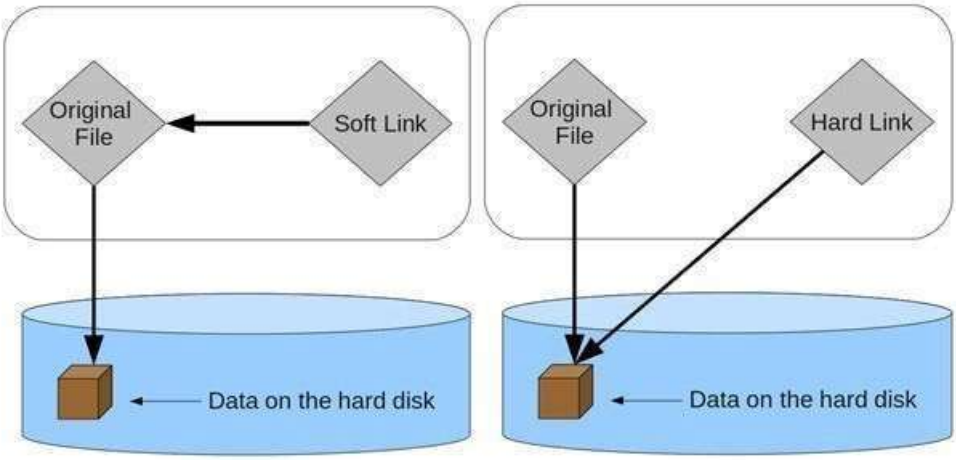
OWASP/SANS Category: A02:2025 – Security Misconfiguration

Description:

Configuration refers to the process of setting up and defining the parameters, options, and settings of a system, software application, or hardware device so that it operates according to specific requirements. It involves adjusting system properties, enabling or disabling features, assigning resources, and establishing rules that control how the system functions. Proper configuration ensures that the system performs efficiently, securely, and in alignment with business or operational needs. It may include configuring network settings, user permissions, security policies, software preferences, and hardware components. Effective configuration management is important for maintaining system stability, preventing errors, and ensuring consistent performance across an organization's IT environment.

Business Impact:

The business impact of configuration is significant because proper configuration ensures that systems, software, and hardware operate efficiently, securely, and according to organizational requirements. Well-configured systems help improve productivity and performance by enabling applications and services to run smoothly with minimal errors or downtime. It also strengthens security, as correct configuration of security settings, access controls, and policies helps protect business data from unauthorized access and cyber threats. In addition, effective configuration management supports operational stability and compliance by ensuring that systems follow standardized settings and regulatory requirements. Poor or incorrect configuration can lead to system failures, vulnerabilities, data loss, or service interruptions, which may result in financial losses, reduced customer trust, and operational delays. Therefore, maintaining proper configuration is essential for reliable IT operations and overall business continuity.



Recommendations:

1. Use NTFS File System:

Hard link configuration should be done only on NTFS file systems because Windows hard links are supported only by NTFS.

2. Ensure Same Volume Configuration:

Both the original file and the hard link must be located in the same disk partition or volume. Hard links cannot be created across different drives.

3. Maintain Clear Directory Organization:

While configuring hard links, files should be placed in well-organized directories so users can easily understand which files are linked.

4. Manage Link Count Properly:

It is recommended to monitor the number of hard links pointing to a file. The file data will remain until the last hard link is removed.

VULNERABILITY NAME: Use of Obsolete Function

CWE/CVE : CWE-447

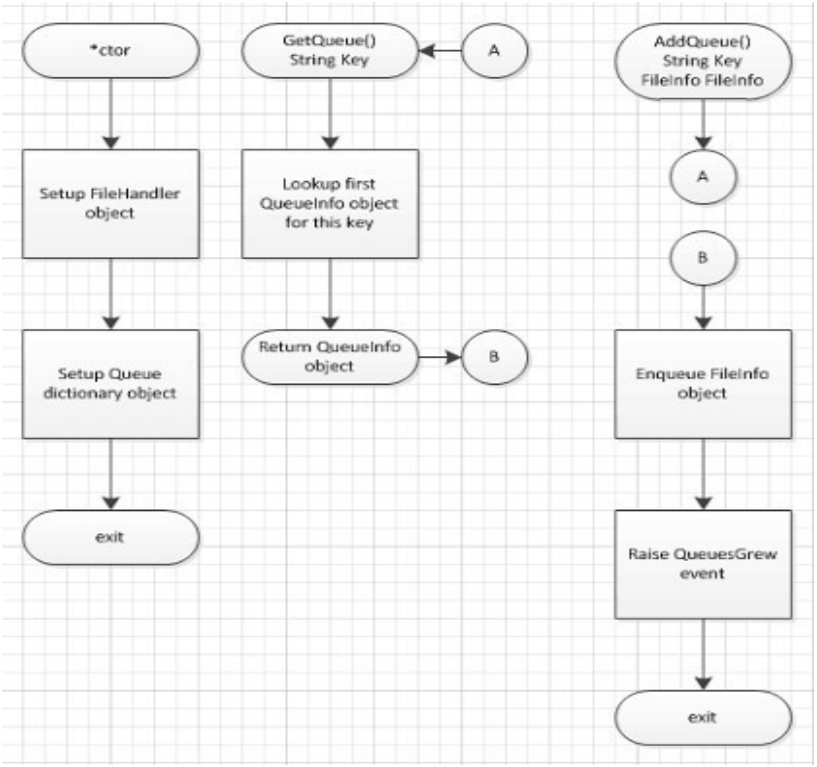
OWASP/SANS Category: A03:2025 – Software Supply Chain Failures

Description:

The use of obsolete functions refers to the practice of using functions, methods, or features in software that have been deprecated or replaced by newer alternatives by the developers or maintainers of a programming language, framework, or application. These functions are typically kept in the system only for backward compatibility with older code, but they are no longer recommended for use because they may be outdated, inefficient, or insecure. Obsolete functions may not receive updates, bug fixes, or security patches, and they may be removed completely in future versions of the software. Therefore, developers are encouraged to replace obsolete functions with the recommended modern alternatives to ensure better performance, maintainability, and long-term compatibility of the software.

Business Impact:

The business impact of using obsolete functions can be significant because outdated functions are often no longer supported or maintained by software vendors. This can lead to security vulnerabilities, as obsolete functions may not receive security updates or patches, making systems more susceptible to cyber threats and attacks. It can also cause system instability or compatibility issues, especially when software is upgraded or integrated with newer technologies that no longer support these outdated functions. In addition, relying on obsolete functions can increase maintenance costs and development time, since developers may need extra effort to troubleshoot issues or rewrite code in the future. If the obsolete function is removed in a newer version of the software, it may lead to application failures or service disruptions, which can impact business operations and productivity. Therefore, continuing to use obsolete functions may reduce system reliability and create long-term technical debt for organizations.



Recommendations:

1. **Replace Deprecated Functions:**

   Developers should avoid using obsolete or deprecated functions and replace them with modern and secure alternatives provided by updated libraries.

2. **Follow Updated Programming Standards:**

   Always follow latest programming standards and documentation to ensure that functions used in the program are supported and secure.

3. **Perform Regular Code Reviews:**

   Code should be reviewed periodically to detect and remove obsolete functions that may introduce vulnerabilities.

VULNERABILITY NAME: Small Space of Random Values

CWE/CVE : CWE-334

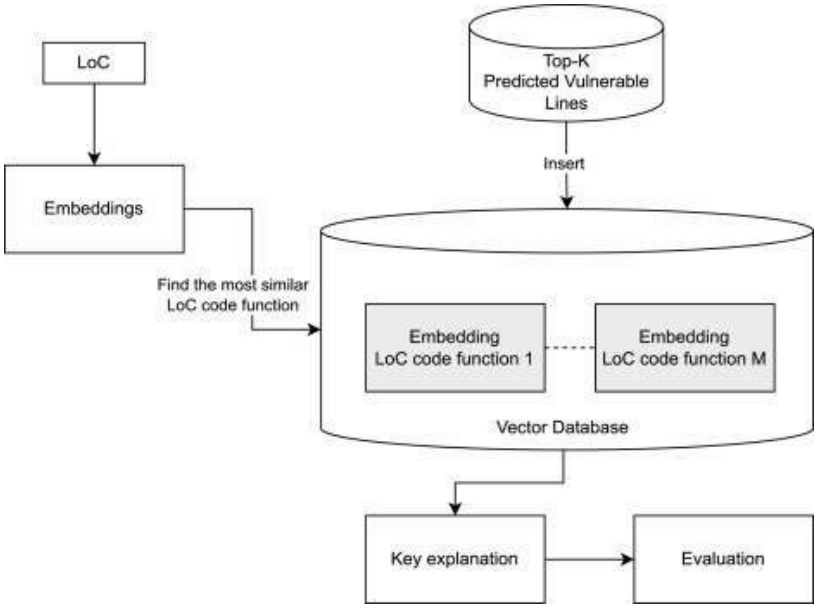
OWASP/SANS Category: A04:2025 – Cryptographic Failures

Description:

Small space random values refers to the use of random numbers or values generated from a limited or small range of possible values. When the range of possible values is small, it becomes easier for attackers or unauthorized users to predict or guess these values, especially in securityrelated processes such as session IDs, authentication tokens, passwords, or encryption keys. Because the number of possible combinations is limited, the randomness is weaker and can be compromised through techniques like brute-force attacks or pattern analysis. Therefore, using small space random values can reduce the effectiveness of security mechanisms and is generally discouraged in secure system design, where larger and more unpredictable value spaces are preferred to ensure stronger protection.

Business Impact:

The business impact of using a small space of random values can be serious because it weakens the security of systems that rely on randomness for protection. When the range of random values is small, attackers can more easily predict or guess values such as session IDs, authentication tokens, or encryption keys. This increases the risk of unauthorized access, account takeover, or data breaches, which can compromise sensitive business and customer information. Such security weaknesses may lead to financial losses, operational disruptions, and damage to the organization's reputation. Additionally, if sensitive data is exposed due to predictable random values, the organization may face legal penalties or regulatory compliance issues. Therefore, using a sufficiently large and secure random value space is important to protect business systems and maintain customer trust.



Recommendations:

1. Use Strong Random Number Generators:

Use cryptographically secure random number generators (CSPRNG) instead of simple random functions.

2. Increase the Random Value Space:

Generate random values from a large range (large key space) so attackers cannot guess them easily.

3. Use Secure Libraries:

Use secure libraries such as:

- SecureRandom in Java
- secrets module in Python
- CryptGenRandom or BCryptGenRandom in Windows

4. Avoid Predictable Seeds:

Do not initialize random generators with predictable values like system time, which attackers can guess.

VULNERABILITY NAME: Process Control

CWE/CVE : CWE- 114

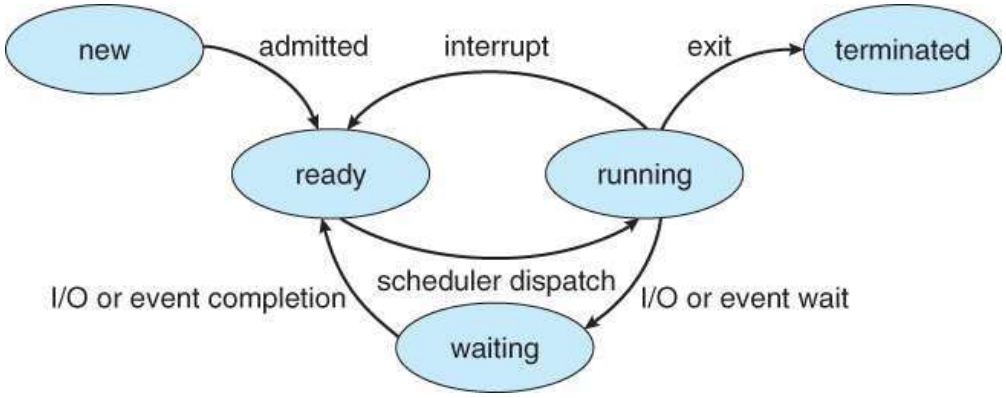
OWASP/SANS Category: A05:2025 – Injection

Description:

Process control refers to the management and regulation of running processes within a computer system to ensure they operate correctly, securely, and efficiently. A process is an instance of a program that is being executed, and process control involves controlling how these processes start, run, communicate, and terminate. It includes mechanisms such as assigning permissions, managing system resources like CPU and memory, and restricting unauthorized access between processes. Effective process control helps maintain system stability, prevents conflicts between programs, and ensures that only authorized processes perform specific operations within the system.

Business Impact:

The business impact of process control is significant because effective management of system processes ensures security, stability, and efficiency in IT operations. Proper process control prevents unauthorized or malicious processes from running, which reduces the risk of data breaches, system compromise, or downtime. It also ensures that critical applications and services receive the necessary system resources, supporting operational efficiency and consistent performance. Poor process control can lead to resource conflicts, application failures, or security vulnerabilities, potentially causing financial losses, productivity disruptions, and reputational damage. For businesses that rely on continuous IT operations, strong process control is essential for maintaining service reliability, compliance with regulations, and safeguarding sensitive information.



Recommendations:

1. Use Proper Process Scheduling:

Implement efficient CPU scheduling algorithms (such as Round Robin, FCFS, or Priority Scheduling) to ensure fair CPU allocation.

2. Maintain Process Control Blocks (PCB):

The operating system should maintain accurate PCB information for each process to manage process execution effectively.

3. Implement Synchronization Mechanisms:

Use synchronization tools like semaphores, mutexes, and monitors to avoid race conditions when multiple processes access shared resources.

4. Ensure Proper Resource Allocation:

Processes should be given adequate CPU time, memory, and I/O resources to prevent starvation and deadlocks.

VULNERABILITY NAME: Excessive Attack Surface

CWE/CVE : CWE- 1125

OWASP/SANS Category: A06:2025 –Insecure Design

Description:

Excessive attack surface refers to a situation where a system, application, or network has more entry points, features, or exposed components than necessary, increasing the potential ways an attacker can exploit it. Each additional service, port, API, or unnecessary functionality adds to the attack surface, providing more opportunities for security breaches. A large or poorly managed attack surface makes it harder to monitor and secure all possible vulnerabilities, thereby increasing the system's exposure to threats such as malware, unauthorized access, or data theft. Reducing the attack surface through proper system design, removing unnecessary features, and limiting exposure is a key aspect of cybersecurity risk management.

Business Impact:

The business impact of an excessive attack surface can be substantial, as it increases the organization's exposure to cyber threats and potential security breaches. A larger attack surface provides more entry points for attackers, making it easier for them to exploit vulnerabilities in applications, networks, or systems. This can lead to data breaches, theft of sensitive information, financial losses, and operational disruptions. Additionally, managing and securing a wide attack surface requires more resources, monitoring, and maintenance, which increases IT costs and complexity. Organizations may also face reputational damage and regulatory penalties if a breach occurs due to insufficient control over exposed components. Reducing the attack surface through streamlined system design, removal of unnecessary features, and robust security practices is therefore critical to protecting business assets and maintaining trust with customers and partners.

Recommendations:

1. Remove Unnecessary Services:

Disable or uninstall unused software, services, and applications that are not required for system operation.

2. Close Unused Ports:

Regularly scan systems and close open ports that are not required to prevent unauthorized access.

3. Apply the Principle of Least Privilege:

Ensure users and applications only have minimum permissions required to perform their tasks.

4. Regular Patch Management:

Keep operating systems, applications, and firmware updated with the latest security patches.

VULNERABILITY NAME: Use of Default Password

CWE/CVE : CWE- 1393

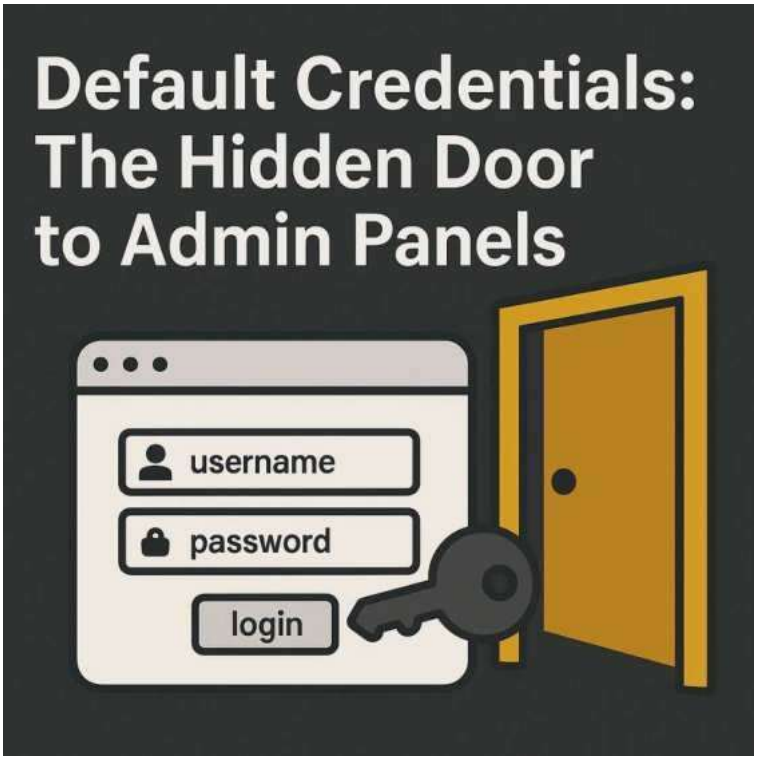
OWASP/SANS Category: A07:2025 - Authentication Failures

Description:

The use of default passwords refers to the practice of leaving the pre-configured passwords that come with hardware devices, software applications, or systems unchanged. Manufacturers and software vendors often assign default passwords to simplify initial setup, but these passwords are typically well-known or easily discoverable, making systems vulnerable to unauthorized access. Using default passwords poses a serious security risk because attackers can exploit them to gain control over devices, access sensitive data, or compromise networks. Best practice requires changing default passwords to strong, unique credentials immediately after installation to protect systems from potential breaches.

Business Impact:

The business impact of using default passwords can be severe, as it significantly increases the risk of unauthorized access and security breaches. Attackers often exploit default credentials to gain control of systems, steal sensitive data, or disrupt operations. Such breaches can lead to financial losses, regulatory penalties, and legal liabilities, especially if customer or proprietary data is exposed. In addition, compromised systems can cause operational downtime, loss of productivity, and reputational damage, eroding customer trust and business credibility. Managing systems with default passwords also increases the burden on IT teams to detect and remediate vulnerabilities. Ensuring strong, unique passwords for all systems is therefore critical for protecting business assets and maintaining secure, reliable operations.



Recommendations:

1. Change Default Credentials Immediately:

After installing any system or device, change the default username and password to a strong, unique password.

2. Use Strong Password Policies:

Create passwords that include:

- Uppercase and lowercase letters
- Numbers

3. Implement Multi-Factor Authentication (MFA):

Add an extra verification step such as OTP, fingerprint, or authentication apps to increase security.

VULNERABILITY NAME: Embedded Malicious Code

CWE/CVE : CWE- 506

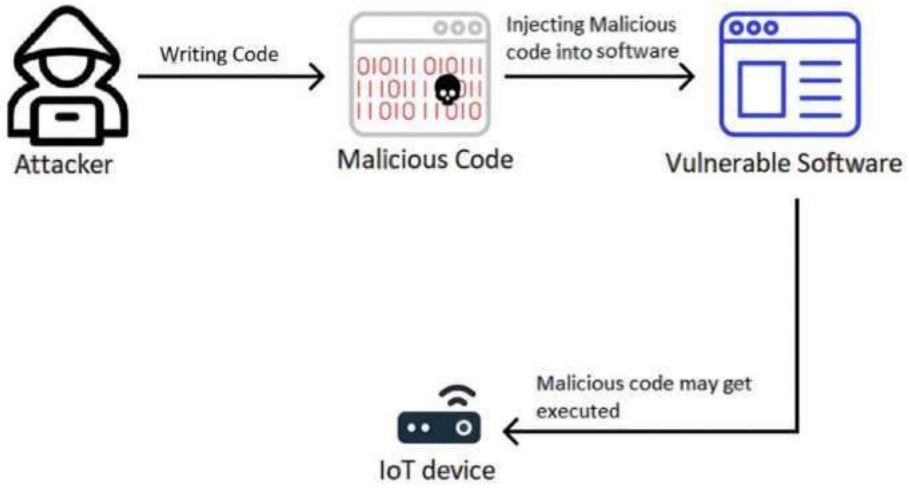
OWASP/SANS Category: A08:2025 – Software or Data Integrity Failures

Description:

Embedded malicious code refers to harmful software, scripts, or instructions that are intentionally inserted into legitimate programs, files, or systems to perform unauthorized or destructive actions. This code can be hidden within applications, documents, firmware, or websites, often going unnoticed by users or security software. Once executed, embedded malicious code can steal data, disrupt operations, create backdoors, or spread malware to other systems. Because it is integrated into otherwise legitimate software, detecting and removing it can be challenging, making it a serious threat to system security and data integrity.

Business Impact:

The business impact of embedded malicious code can be severe, as it can compromise system security, data integrity, and operational continuity. Once activated, such code can steal sensitive information, disrupt critical applications, or create backdoors that allow attackers ongoing access to corporate systems. This can result in financial losses, regulatory penalties, and legal liabilities, especially if customer or proprietary data is exposed. Additionally, embedded malicious code can cause system downtime, reduced productivity, and reputational damage, eroding trust with customers, partners, and stakeholders. Detecting and mitigating it often requires significant IT resources and incident response efforts, increasing operational costs. Businesses therefore need strong security measures, including code scanning, monitoring, and secure software development practices, to minimize the risks associated with embedded malicious code.



Recommendations:

1. **Use Trusted Software Sources:**

   Download applications only from official websites or trusted app stores to reduce the risk of infected software.

2. **Implement Code Review and Testing:**

   Developers should perform regular code reviews and security testing to detect hidden or suspicious code.

3. **Use Antivirus and Anti-Malware Tools:**

   Install and update security software to detect and remove malicious code.

4. **Apply Secure Coding Practices:**

   Developers should follow secure programming standards to avoid vulnerabilities that allow malicious code insertion.

VULNERABILITY NAME: Insufficient Logging

CWE/CVE : CWE- 778

OWASP/SANS Category: A09:2025 – Security Logging and Alerting Failures

Description:

Insufficient logging refers to a situation where a system, application, or network does not record enough information about its activities, events, or transactions to provide a clear and comprehensive audit trail. Logging is crucial for tracking user actions, system errors, security events, and operational processes. When logging is insufficient, it becomes difficult to detect, investigate, or respond to security incidents, performance issues, or compliance violations. This can hinder forensic analysis after a breach, obscure the root cause of system failures, and limit the ability to monitor for suspicious or unauthorized activity. Effective logging should capture the right level of detail while balancing storage and privacy considerations.

Business Impact:

The business impact of insufficient logging can be significant because it reduces an organization's ability to detect, investigate, and respond to security incidents or operational issues. Without detailed logs, businesses may fail to identify breaches, unauthorized access, or system failures in a timely manner, increasing the risk of data loss, prolonged downtime, and regulatory non-compliance. Insufficient logging also hampers forensic investigations, making it difficult to determine the cause of incidents or hold parties accountable, which can lead to legal liabilities and reputational damage. Additionally, inadequate logs can prevent organizations from monitoring system performance effectively, potentially resulting in reduced operational efficiency and higher IT costs. Proper logging is therefore essential for security, compliance, and business continuity.

Recommendations:

1. Implement Comprehensive Logging:

Ensure systems log important events such as:

- User logins and logouts
- Failed login attempts
- File access and modifications

2. Enable Real-Time Monitoring:

Use Security Information and Event Management (SIEM) or monitoring tools to analyze logs in real time and detect suspicious activities.

3. Protect Log Files:

Secure log files by restricting access and preventing unauthorized modification or deletion.

VULNERABILITY NAME: Divide By Zero

CWE/CVE : CWE- 369

OWASP/SANS Category: A10:2025 – Mishandling of Exceptional Conditions

Description:

Divide by zero refers to a computational error that occurs when a program attempts to divide a number by zero. In mathematics, division by zero is undefined, and in computing, performing such an operation can cause a program to crash, produce exceptions, or generate unpredictable results. This error can occur in software applications, scripts, or systems when input validation is missing or calculations do not account for the possibility of zero as a divisor. Handling divide-by-zero scenarios properly—through input checks, exception handling, or alternative logic—is essential to maintain system stability and prevent runtime failures.

Business Impact:

The business impact of divide-by-zero errors can be substantial, as such errors can cause software crashes, system failures, or incorrect calculations, potentially disrupting business operations. In critical applications—such as financial systems, inventory management, or data analytics—divide-by-zero errors can lead to inaccurate reports, wrong decision-making, and operational delays. zsIf these errors occur in customer-facing systems, they can result in poor user experience, loss of trust, and reputational damage. Additionally, resolving such errors often requires developer time and IT resources, increasing operational costs. Ensuring proper input validation and exception handling is therefore essential to maintain system reliability, accuracy, and business continuity.

Recommendations:

1. Validate Input Values:

Always check input values before performing division operations to ensure the divisor is not zero.

2. Implement Error Handling:

Use exception handling mechanisms (try-catch blocks) to prevent program crashes when such errors occur.

3. Apply Input Validation:

Ensure that user inputs or external data are validated and sanitized before being used in calculations.

4. Use Defensive Programming:

Write code that anticipates possible errors and handles them safely to avoid system failures.

Stage-2

Overview :

Nessus is a widely used vulnerability assessment tool developed by Tenable that helps organizations identify security weaknesses in their networks, systems, and applications. It is designed to scan computers, servers, databases, web applications, and network devices to detect vulnerabilities that could be exploited by attackers. Nessus uses a large and frequently updated database of vulnerability plugins to check for known security issues such as missing patches, outdated software versions, insecure configurations, open ports, weak encryption protocols, and exposed services. The tool performs automated scans on specified targets and analyzes the system for potential risks. After the scan is completed, Nessus generates detailed reports that categorize vulnerabilities based on severity levels such as informational, low, medium, high, and critical. These reports also include explanations of the vulnerabilities, their potential impact, and recommendations for remediation or mitigation.

In addition, Nessus supports both authenticated and unauthenticated scanning. Authenticated scans allow the tool to log into the system using credentials and perform deeper analysis, which results in more accurate vulnerability detection. Nessus also helps organizations comply with various security standards and regulations by identifying configuration weaknesses and policy violations. Security analysts, penetration testers, and IT administrators commonly use Nessus as part of their vulnerability management process to regularly monitor systems, prioritize security risks, and fix vulnerabilities before they can be exploited. Because of its reliability, comprehensive vulnerability coverage, and user-friendly interface, Nessus has become one of the most trusted and widely used vulnerability scanning tools in the field of cybersecurity.

Target website: testfire.net

Target IP address: 54.83.41.36

List of vulnerability :

S.No	Vulnerability name	Severity	plugins
1	Cross-site scripting (stored)	Low	Web application Scanner
2	Broken Access Control	Medium	SSH weak HMAC Algorithms Supported
3	SQL Injection	Medium	N/A
4	Broken Authentication	Low	HSTS Missing from Server
5	Insecure Direct Object Reference	Info	Service Detection
6	Security Misconfiguration	Low	SSL cipher Block Chaining cipher suite
7	Cross-Site Request Forgery	Low	HTTP Form- Based Authentication Detected
8	Cleartext Transmission of Sensitive Information	Medium	Cleartext Transmission of Sensitive Information over HTTP
9	ClickJacking	Info	HTML Rendering Layer Vulnerability
10	webserver allows password auto- completion	Low	SSL/TLS Diffie-Hellman Modulus<=1024 bits(Logjam)

VULNERABILITY NAME: Cross site scripting (stored)

SEVERITY : Low

PLUGIN: Web application scanner

PORT:N/A

Description:

It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser.

Solution:

- When interacting with databases, use parameterized statements or prepared statements to Avoid SQL injection, which can be a vector for XSS.

  Sanitize and validate all user inputs and ensure that any data displayed on the web page properly encoded.
- Use output encoding libraries or functions to prevent script injection.

Business impact:

The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area that is displayed to either many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data .

VULNERABILITY NAME: Broken access control
SEVERITY: Medium
PLUGIN: SSH weak HMAC Algorithms supported
PORT: N/A

Description:

When access control checks are not applied consistently - or not at all users are able to access data or perform actions that they should not be allowed to perform. This can lead to a wide range of problems, including information exposures, denial of service, and arbitrary code execution.

Solution:

- Ensure that the application has a robust authentication and authorization mechanism in place.
- Users should be authenticated before any sensitive actions are performed, and authorization checks should be conducted to verify that users have the necessary permissions for the requested operation.
- Ensure that session management is secure. Use strong session IDs, enforce session timeouts, and regenerate session tokens after login. Implement session fixation protection to prevent session hijacking.

Business impact:

Wide-ranging business impact, including financial losses, reputation damage legal consequences, operational disruption, and loss of customer trust. Addressing this vulnerability is crucial to protect the organization and its stakeholders from these potential negative effects.

VULNERABILITY NAME: SQL injection

SEVERITY : Medium

PLUGIN: N/A (typically would be a security testing tool)

PORT: Port 80/443

Description:

The product constructs all or part of an SQL command using externally influenced input from an upstream component, but it does not neutralize or incorrectly neutralizes special elements that could modify the intended SQL command when it is sent to a downstream component.

Solution:

- If parameterized statements are not feasible, use proper input validation and escaping mechanisms to sanitize user inputs before they are used in SQL queries. This helps prevent malicious code injection.
- Utilize stored procedures for database operations. This can help separate SQL code from application code and reduce the risk of SQL injection.
- Implement strict input validation for all user inputs, ensuring that data adheres to the expected format and structure.

Business imapct:

In summary, it is crucial to underscore that CWE-89, known as SQL Injection,can exert a profound and diverse business impact. This encompasses critical facets such data breaches, financial setbacks, harm to reputation, legal ramifications, and operational turmoil.

VULNERABILITY NAME: Broken authentication

SEVERITY: Low

PLUGIN: HSTS Missing from HTTPS Server (plugin id:142960)

PORT: Port 443

Description:

The system's authorization functionality does not prevent one user from gaining access to another user's data or record by modifying the key value identifying the data.

Solution:

- Implement strong authentication mechanisms, such as multi-factor authentication (MFA or two-factor authentication (2FA), to ensure that only authorized users can access the system.
- Implement proper authorization controls to ensure that users have the necessary permissions to access specific functions or data. Use role-based access control (RBAC) to manage user privileges effectively.
- Deploy intrusion detection and prevention systems (IDS/IPS) to monitor and block suspicious activities related to authentication and authorization.

Business impact:

To effectively mitigate the business impact of CWE-285, it is imperative that organizations place a strong emphasis on bolstering their authentication and session management practices. This should encompass the adoption of multi-factor authentication, the secure storage of credentials, meticulous session handling, and a commitment to conducting routine security assessments and testing.

VULNERABILITY NAME: Insecure Direct object Reference
SEVERITY: Info
PLUGIN: Service Detection (plugin id:22964)
PORT: Port 80/443

Description:

Insecure Direct Object Reference (IDOR) is a vulnerability that arises whenattackers can access or modify objects by manipulating identifiers used in a web application'sURLs or parameters. It occurs due to missing access control checks, which fail to verify whether a user should be allowed to access specific data.

Solution:

- Utilize parameterized queries or prepared statements when interacting with databases. This prevents user-controlled input from being executed as SQL code.
- Implement a Content Security Policy to mitigate the risk of cross-site scripting (XSS) attacks, which can be used to execute unauthorized queries.
- Implement account lockout mechanisms that temporarily suspend accounts after a specified number of unauthorized access attempts to prevent brute force attacks.

Business impact:

To effectively mitigate the business impact of CWE-639, organizations must make it a top priority to fortify their access control and authorization mechanisms. ThiSentails the implementation of robust security measures, the regular conduct of comprehensive security assessments, and the deployment of intrusion detection systems to promptly identify and counter unauthorized access attempts. These measures stand as critical in the defence of sensitive data, the overall security of systems, and the preservation of the organization's esteemed reputation.

VULNERABILITY NAME: Security Misconfiguration
SEVERITY: low
PLUGIN: SSL cipher Block chaining cipher suites
PORT: Port 443

Description:

The product specifies permissions for a security-critical resource in a way that allows that resource to be read or modified by unintended actors.

Solution:

- Implement a change management process to document and review all changes to system configurations. This helps prevent unauthorized or unintended alterations.
- Follow security guidelines provided by software and hardware vendors to ensure secure configurations.
- Regularly perform compliance checks against industry standards and regulations to ensure that configurations remain in compliance with security requirements.

Business impact:

To effectively lessen the business consequences stemming from CWE-732, organizations must place a strong emphasis on bolstering their configuration management practices. This involves the establishment of secure default settings, the routine conduct of comprehensive security evaluations, and the deployment of intrusion detection systems to promptly detect and address misconfigurations and vulnerabilities. These actions are of paramount importance in safeguarding sensitive data, fortifying system security, and preserving the organization's esteemed reputation.

VULNERABILITY NAME: Cross-Site Request Forgery
SEVERITY: Low
PLUGIN: HTTP Form-Based Authentication Detected
PORT: Port 8080/443

Description:

Cross-Site Request Forgery (CSRF) is an attack that forces an end user to executeunwanted actions on a web application in which they're currently authenticated. With a little helpof social engineering (such as sending a link via email or chat), an attacker may trick the users of a web application into executing actions of the attacker's choosing. If the victim is a normal user, a successful CSRF attack can force the user to perform state changing requests like transferring funds, changing their email address, and so forth. If the victim is an administrative account, CSRF can compromise the entire web application.

Solution:

- Implement Synchronizer Token Patterns to include unique, randomly generated tokens in each HTTP request and validate these tokens on the server side to verify the authenticity of the request.
- Utilize the Same Site attribute in cookies to restrict their usage to same-site requests only, preventing them from being sent along with cross-site requests and effectively mitigating CSRF attacks.

Business impact:

To effectively reduce the business repercussions of CWE-352, organization must prioritize secure coding practices, including the incorporation of anti-CSRF tokens, while also conducting routine security assessments and testing to detect and rectify CSRF vulnerabilities. Furthermore, educating users on secure browsing practices is pivotal in preventing CSRF attacks. These actions are of paramount importance in the protection of data and in upholding the trust of customers and partners.

VULNERABILITY NAME: Cleartext Transmission of Sensitive Information
SEVERITY: Medium
PLUGIN: Cleartext Transmission of Sensitive Information over HTTP
PORT: Port 443

Description:

The product transmits sensitive or security-critical data in cleartext in a communication channel that can be sniffed by unauthorized actors. Many communication channels can be "sniffed" (monitored) by adversaries during data transmission. For example, in networking, packets can traverse many intermediary nodes from the source to the destination, whether across the internet, an internal network, the cloud, etc. Some actors might have privileged access to a network interface or any link along the channel, such as a router, but they might not be authorized to collect the underlying data. As a result, network traffic could be sniffed by adversaries, spilling securitycritical data.

Solution:

- Educate users and employees on secure data handling practices, such as recognizing secure websites (look for "https") and avoiding insecure Wi-Fi networks.
- Employ data masking or redaction techniques to replace sensitive data with placeholders or cryptographic representations during transmission.

Business impact:

To effectively reduce the business consequences associated with CWE-319,organizations must prioritize the adoption of secure data transmission practices. This includes the utilization of encryption and robust, secure protocols. The routine conduct of security assessments and testing is pivotal in pinpointing and remedying vulnerabilities linked to data transmission. Furthermore, the education of users on secure data handling practices plays a vital role in proactively preventing data exposure incidents. These actions are of paramount importance in the protection of sensitive data and in preserving the trust of both customers and partners.

VULNERABILITY NAME: Clickjacking
SEVERITY: Info
PLUGIN: HTML Rendering Layer Vulnerability
PORT: Port 80/443

Description:

The web application does not restrict or incorrectly restricts frame objects or UI layers that belong to another application or domain, which can lead to user confusion about which interface the user is interacting with.

Solution:

- Set the X-Frame-Options HTTP response header to deny or same-origin to control how your site can be framed. This is supported by most modern browsers.
- Utilize frame-busting JavaScript code in web applications to prevent the embedding of your site within malicious frame's. This code can disrupt clickjacking attempts.
- Implement a Content Security Policy to restrict which domains can embed your site in iframes. This can help prevent unauthorized framing.
- Implement additional security controls to prevent UI redress attacks, such as clickjacking, within your web application.

Business impact:

To effectively reduce the business consequences resulting from CWE-1021,organizations must prioritize the implementation of protective measures, such as frame-busting code. Simultaneously, educating users on safe browsing practices plays a crucial role in preventing clickjacking incidents. Additionally, routine security assessments and testing are pivotal for identifying and mitigating vulnerabilities associated with clickjacking. These actions are of paramount importance in upholding user trust, ensuring data protection, and safeguarding the organization's reputation.

VULNERABILITY NAME: Web server allows password auto-completion
SEVERITY: Low
PLUGIN: SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)
PORT: Port 443

Description:

Weaknesses in this category are related to the design and implementation of data confidentiality and integrity. Frequently these deal with the use of encoding techniques, encryption libraries, and hashing algorithms. The weaknesses in this category could lead to a degradation of the quality data if they are not addressed.

Solution:

- Implement secure password storage mechanisms, such as using strong and salted cryptographic hashing algorithms like crypt or script.
- Implement robust encryption key management practices, including secure key storage, key rotation, and access controls.

Business Impact:

To effectively lessen the business consequences associated with CWE-310, organizations must prioritize the adoption of secure cryptographic practices. This entails ensuring proper password storage and robust encryption key management. Simultaneously, conducting routine security assessments and testing is pivotal in detecting and mitigating vulnerabilities related to cryptographic issues. Striving for compliance with pertinent data protection regulations and industry standards is equally essential. These actions are of paramount importance in the protection of sensitive data, the upholding of user trust, and the preservation of the organization's esteemed reputation.

Target Website: vit.ac.in

Target IP address: 122.184.65.22

List of vulnerability :

S.No	Vulnerability Name	Severity	Plugins
1	Cross-site scripting (DOM based XSS)	Medium	Multiple XSS
2	jQuery 1.3<3.5.0	Medium	CGI Abuse: XSS
3	information exposure (web server HTTP header internal IP disclosure)	Low	Webserver HTTP Header Internal IP Disclosure
4	Information exposure (SSL (Multiple Issues))	Info	SSL Certificate Information
5	Information exposure (HTTP (Multiple Issues))	Info	HTTP Information
6	OS identification failed	Info	TCP/IP Remote Probing

REPORT:

VULNERABILITY NAME: Cross-site scripting (DOM based XSS)

SEVERITY: Medium

PLUGIN: Multiple XSS

PORT: Port 443

Description:

It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser. DOMXSS: DOM stands for Document Object Model and is a programming interface for HTML and XML documents. It represents that programs can change the document structure, style and content. A webpage is a document, and this document can be either displayed in the browser window or as the HTML Source.

Solution:

- When interacting with databases, use parameterized statements or prepared statements to avoid SQL injection, which can be a vector for XSS.
- Consider using well-established JavaScript frameworks and libraries that include built-in security features to prevent DOM XSS.

Business impact:

The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area displayed to many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.

VULNERABILITY NAME: jQuery 1.2 < 3.5.0 Multiple XSS
SEVERITY: Medium
PLUGIN: CGI Abuse: XSS
PORT: Port 443

Description:

It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser. Stored XSS (Persistent XSS): In this type of attack, the malicious script is permanently stored on a server or in a database, often in user-generated content like comments or forum posts. When other users view the affected content, the script executes in their browsers, potentially compromising their data or sessions. Reflected XSS: Reflected XSS occurs when the malicious script is embedded in a URL and immediately executed when a victim clicks on a manipulated link.

Solution:

- The most important recommendation is to update to the latest version of jQuery. Newer versions typically include security patches, bug fixes, and performance improvements.
- Stay vigilant about jQuery updates and security announcements. Subscribe to relevant security mailing lists or forums to receive notifications about new releases.

Business impact:

The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area displayed to many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.

VULNERABILITY NAME: Information Exposure (Web Server HTTP Header Internal IP Disclosure)
SEVERITY: Low
PLUGIN: Web Server HTTP Header Internal IP Disclosure
PORT: Port 80

Description:

The product exposes sensitive information to an actor that is not explicitly authorized to have access to that information.

Solution:

- Carefully review your web server's configuration files (e.g., Apache's httpd.conf, Nginx's nginx.conf) to identify anywhere internal IP addresses are inadvertently included in HTTP response headers.
- Remove or replace any internal IP addresses in the server configuration files with appropriate placeholders or public IP addresses.
- If you use proxy servers or load balancers, ensure that they are configured to hide internal IP addresses and only expose public IP addresses instances in HTTP headers.
- Review and configure HTTP response headers (e.g., Server, X-Powered-By) to limit the exposure of server and technology details, reducing the potential for disclosing internal IP addresses.

Business Impact:

To effectively reduce the business consequences arising from SSL issues, organizations must give utmost priority to establishing a secure SSL/TLS configuration, conducting frequent security assessments and testing. Adhering to data protection regulations and industry standards is imperative. These actions play a pivotal role in safeguarding sensitive data, upholding user trust, and safeguarding the organization's esteemed reputation.

VULNERABILITY NAME: Information Exposure (SSL (Multiple Issues))
SEVERITY: Info
PLUGIN: SSL certificate Information
PORT: Port 443

Description:

The product exposes sensitive information to an actor that is not explicitly authorized to have access to that information.

Solution:

- Configure your web server to use the latest and strongest encryption protocols, such as TLS 1.2 or higher. Disable outdated and insecure protocols like SSLv3.
- Implement Perfect Forward Secrecy to ensure that even if the private key is compromised, previously recorded encrypted traffic remains secure.
- Safeguard your private keys by storing them securely and ensuring they are not accessible to unauthorized parties.

Business impact:

To effectively reduce the business consequences arising from SSL issues, organizations must give utmost priority to establishing a secure SSL/TLS configuration, conducting frequent security assessments and testing. Adhering to data protection regulations and industry standards is imperative. These actions play a pivotal role in safeguarding sensitive data, upholding user trust, and safeguarding the organization's esteemed reputation.

VULNERABILITY NAME: Information Exposure (HTTP (Multiple Issues))
SEVERITY: Info
PLUGIN: HTTP information
PORT: Port 80 /443

Description:

The product exposes sensitive information to an actor that is not explicitly authorized to have access to that information

Solution:

- Minimize the use of unnecessary redirects as they can increase page load times. Use 301 permanent) redirects for consistent content.
- Regularly check for broken links (404 errors) and fix them to ensure users find the content they are looking for.
- Properly configure CORS to control which domains are allowed to make requests to your server.
- Migrate your website from HTTP to HTTPS to ensure secure data transmission. This is essential for user trust and search engine ranking.

Business impact:

To effectively reduce the business repercussions of HTTP issues, organizations must place a strong emphasis on ongoing monitoring and enhancement of their web services. Consistent performance testing and security assessments, coupled with proactive maintenance, are indispensable to ensure a seamless and secure online presence. Furthermore, offering timely and helpful customer support and transparent communication when HTTP issues arise is instrumental in mitigating adverse effects on user trust and satisfaction.

VULNERABILITY NAME: OS Identification Failed
SEVERITY: Info
PLUGIN: TCP/IP remote probing
PORT: N/A

Description:

OS identification failed refers to the inability of a system or software to accurately recognize the operating system running on a device or network. This issue can result from various factors such as misconfigurations, network issues, or compatibility problems. The failure to identify the OS correctly can lead to security vulnerabilities, hindered software updates, and potential operational disruptions. Resolving this issue promptly is crucial to ensure accurate security measures and the smooth functioning of the system or network.

Solution:

- Minimize the use of unnecessary redirects as they can increase page load times. Use 301 (permanent) redirects for consistent content.
- Regularly check for broken links (404 errors) and fix them to ensure users find the content they are looking for.
- Properly configure CORS to control which domains are allowed to make requests to your server.
- Migrate your website from HTTP to HTTPS to ensure secure data transmission. This is essential for user trust and search engine ranking.

Business impact:

It may lead to operational inefficiencies, potential security vulnerabilities, non-compliance with industry regulations, and a possible loss of customer trust. Such failures could result in downtime, compromised data security, regulatory penalties, and damage to the company's reputation, potentially leading to financial losses and decreased market competitiveness. Resolving this issue promptly is crucial to mitigate these risks and maintain the smooth functioning of the organization.

Stage-3

Title:

Design and apply context-aware access controls based on user identity, device posture, location, and application risk.

Context-aware access controls represent the operational pinnacle of a Zero Trust architecture, moving beyond static rules to create a dynamic and adaptive security perimeter around every application. In a modern enterprise, where users are mobile, devices are diverse, and threats are sophisticated, granting access based solely on a username and password is a dangerous oversimplification. Designing and applying context-aware controls means building a policy engine that consumes multiple, real-time signals about an access request and renders an intelligent decision: to allow, to block, to limit, or to challenge. This approach ensures that the security posture is not a flat, always-on gate but a responsive system that tightens or loosens based on the risk of the specific moment.

The design of such a system begins with a deep integration between the access broker, such as Zscaler Private Access (ZPA), and the organization's existing security and identity infrastructure. The first and most fundamental layer is user identity. This is not simply a username, but a rich set of attributes pulled from the corporate Identity Provider (IdP) like Azure AD or Okta. Integration via SAML ensures that when a user authenticates through the ZPA client, they are verified with multi-factor authentication (MFA) by the IdP. ZPA then ingests not just the user's name, but their group memberships (e.g., "Finance – Payroll Managers," "Engineering – Source Code Contributors"), roles, and department. This allows for the creation of granular policies that are intrinsically tied to the user's function within the organization.

For instance, a policy can be written in plain language: "Allow members of the 'Finance-Payroll' group to access the 'ADP Payroll' application." However, a valid identity on a compromised device is still a critical threat. This is where the second layer, device posture, becomes essential. The design incorporates continuous checks on the security health of the endpoint attempting the connection. Through integrations with Endpoint Detection and Response (EDR) tools like

CrowdStrike or SentinelOne, and Mobile Device Management (MDM) solutions like JAMF or Intune, ZPA can assess a device's real-time hygiene. The policy engine evaluates factors such as: Is the device's disk encryption (BitLocker/FileVault) enabled? Is the antivirus software active and its definition database up-to-date? Has the operating system been patched to the latest security baseline? Is the device rooted or jailbroken? Is there a high-risk score being reported by the EDR tool due to suspicious activity? If a device fails these checks—for example, a user's laptop has its firewall disabled—the context-aware policy can automatically trigger an action. It might block access entirely, or it might grant "remediation access," routing the user to a captive portal to download updates or contact IT, while preventing them from reaching any businesscritical applications. This ensures that a vulnerable endpoint cannot become a beachhead for an attacker to move laterally into the corporate environment.

The third contextual layer is location and network environment. The physical or virtual location from which a user connects provides a powerful signal for risk assessment. A finance director accessing the payroll system from their home office in the same city is a low-risk event. The same user attempting to connect from a country with a high prevalence of cybercrime, or from an anonymizing VPN service, should be treated with extreme suspicion. Context-aware policies can be designed to enforce this logic. For example, access to a source code repository containing intellectual property could be restricted to users connecting from a corporate office or a preapproved home IP address range. If a legitimate user is traveling, the policy might allow access but require a step-up authentication, such as a hardware token, or grant only read-only permissions. This layer can also consider the security of the network itself; a connection from a public, unencrypted Wi-Fi hotspot might trigger stricter controls than one from a known, secure corporate network.

The final and often most strategic layer is application risk. Not all applications are created equal, and their access policies should reflect their value and sensitivity. In the design phase, applications are categorized and tagged based on their risk profile. An internal wiki used for cafeteria menus would be classified as low-risk, whereas a customer database, a financial trading platform, or a source code repository would be classified as high or critical risk. This tagging allows policies to be layered and dynamic. A single user might have access to multiple applications, but the controls applied to each session will differ based on the application's risk.

For the low-risk wiki, a valid user identity and a compliant device might be sufficient. However, for the high-risk financial platform, the policy engine might require all four layers to align: the user must be in a specific group, on a fully patched and managed device, connecting from a trusted location, and may even be subject to session recording or time-of-day restrictions. This ensures that security resources and scrutiny are applied proportionally to the actual risk, preventing "blanket" access and ensuring that the most valuable assets have the most robust defenses.

In practice, applying these controls within ZPA means using the policy builder to create conditional statements. A policy might read: "Allow access to 'Application A' if User is in 'Group X' AND Device Posture is 'Compliant' AND Location is 'Not Restricted' AND Application Risk is 'Low'." Alternatively, a more complex policy for a critical application might be: "Allow access to 'Application B' if User is in 'Group Y' AND Device Posture is 'Highly Secure' AND Location is 'Corporate Network' AND Time is 'Business Hours', otherwise, if Location is 'Remote', require Step-Up Authentication." By combining these signals, the organization creates a security posture that is finely tuned, highly resilient, and fundamentally adaptive to the context of every single access attempt, thereby upholding the core Zero Trust principle of "never trust, always verify."

Conclusion:

Stage 1: In conclusion, web application testing is a crucial process that ensures the security, functionality, and reliability of web-based software. It involves a comprehensive evaluation of various aspects, including user interface, functionality, performance, and security to identify and rectify issues before they impact end-users. This process not only enhances the quality of web applications but also bolsters their resilience against potential threats and vulnerabilities. In an age where online services and applications are integral to our daily lives, effective web application testing is essential to deliver a seamless and secure user experience.

Stage 2: The Nessus report for the specific website has highlighted key security insights. It identified vulnerabilities, categorized by severity, with a focus on critical and high-risk issues that need immediate attention. Recommendations for addressing these vulnerabilities include patching, reconfiguration, and other security measures, with a priority on actions that can have the most significant impact. Compliance with industry standards and best practices is considered, ensuring alignment with recognized security benchmarks like OWASP guidelines. The report underscores the need for continuous monitoring, regular updates, and ongoing security efforts to maintain a strong security posture. In summary, the Nessus report offers a clear view of the website's security state, provides guidance on mitigating vulnerabilities, and stresses the importance of sustained vigilance against evolving threats.

Stage 3: The implementation of Zscaler Private Access marks a pivotal shift from a vulnerable, network- centric security model to a resilient, identity-centric Zero Trust architecture. By enforcing least- privilege access and rendering private applications invisible to the internet, this project effectively neutralizes the threats of lateral movement and network-based attacks. The solution not only fortifies the organization's security posture against modern cyber threats but also delivers a seamless and high-performance experience for a distributed workforce. Furthermore, the decoupling of access from the network simplifies IT operations, reduces dependency on legacy VPN infrastructure, and provides granular visibility into user-application interactions. Ultimately, this initiative establishes a future-ready foundation for secure digital transformation..

Future Scope:

Stage 1: Future scope of web application testing

The future scope of web application testing is promising and evolving rapidly, driven by the continuous growth of the internet and web-based applications. Here are some key aspects of its future scope:

▪ Increased Demand: As more businesses and services move online, the demand for web applications continues to grow. This, in turn, increases the demand for web application testing to ensure security, reliability, and functionality.

▪ Advanced Technologies: The complexity of web applications is rising with the integration of advanced technologies like AI, IoT, and blockchain. Testing these cutting-edge applications will require specialized skills and tools.

▪ Security Emphasis: With the growing number of cyber threats and data breaches, security testing for web applications will become even more critical. Testers will need to focus on vulnerabilities, data protection, and compliance with regulations like GDPR.

▪ Mobile and Cross-Browser Testing: Mobile usage is on the rise, and web applications must be responsive and functional on various devices and browsers. Testing for cross-compatibility will be essential.

▪ Performance Testing: As user expectations for speed and performance increase, load testing, stress testing, and other performance testing types will be crucial to ensure web applications can handle traffic spikes.

Stage 2: Future scope of the testing process you understood.

The future scope of the web application testing process, particularly in terms of identifying vulnerabilities, is exceedingly promising due to the continuous growth of digital platforms, the escalating importance of cybersecurity, and the ever-evolving threat landscape. Here are some key aspects that highlight the prospective scope of web application testing for vulnerability discovery:

▪ AI-Driven Testing: The integration of artificial intelligence (AI) and machine learning (ML) into web application testing tools will revolutionize the way vulnerabilities are identified. AI

algorithms can swiftly scan vast amounts of code and data to pinpoint vulnerabilities that might elude human testers.

▪ IoT and Mobile App Testing: With the proliferation of Internet of Things (IoT) devices and the increasing usage of mobile applications, the scope of web application testing will extend to encompass these areas. Ensuring the security of IoT devices and the interfaces they use will become critical.

▪ Blockchain and Cryptocurrency Security: As blockchain technology and cryptocurrencies continue to gain traction, security testing will be imperative to safeguard transactions and digital assets. Ensuring the robustness of blockchain-based applications will be a specialized niche in web application testing. Serverless Architectures: With the adoption of serverless computing, web application testing will need to adapt to assess the security of serverless functions and APIs. Ensuring that serverless components are free from vulnerabilities will be paramount. ▪ Container Security: As containerization technologies like Docker and Kubernetes become ubiquitous, web application testing will extend to cover container security. This includes ensuring that container images and orchestration configurations are free from vulnerabilities.

Stage 3: Future scope of Aware Access Control

The successful deployment of ZPA lays the groundwork for a broader, more integrated security ecosystem. Future enhancements can be explored to further mature the organization's security capabilities:

▪ Integration with Zscaler Internet Access (ZIA): Converging Private Access (ZPA) with Internet Access (ZIA) to create a unified Secure Access Service Edge (SASE) platform, managing all web and private traffic through a single policy framework.

▪ Advanced Device Posture Checking: Moving beyond basic compliance to incorporate realtime risk scores from EDR tools, blocking access if a device shows signs of active compromise.

▪ Privileged Remote Access: Extending ZPA to third-party vendors and contractors with time-bound, audited access to specific critical systems, replacing jump boxes and bastion hosts.

▪ IoT/OT Segmentation: Applying the same micro-tunneling principles to Internet of Things (IoT) and Operational Technology (OT) devices to ensure they can only communicate with specific management servers.

▪ AI-Driven Anomaly Detection: Leveraging machine learning on ZPA logs to detect behavioral anomalies, such as a user accessing an application at an unusual hour or from a strange location, triggering automated MFA challenges or access revocation.

▪ Cloud Workload Protection: Utilizing ZPA to secure connections between containers and microservices in multi-cloud environments (AWS, Azure, GCP), ensuring east-west traffic between workloads is encrypted and authenticated.

▪ Automated Provisioning/Deprovisioning: Enhancing the identity provider integration to automatically revoke application access the moment an employee is terminated or changes roles, ensuring near real-time access control.

▪ Digital Experience Monitoring (DEM): Implementing DEM tools to proactively monitor the end- user experience, identifying network latency or application performance issues before they impact productivity.

Topics Explored:

1. Zscaler
2. Zero trust
3. ZPA
4. VPN
5. MISP
6. College Network Information
7. Deployment of SOC in the College
8. Threat Intelligence
9. Incident Response

Tools Explored:

Nessus Scanner